DevOps Interview Question Guide

Table of Contents

1. Interview Timeline & Structure
2. Baseline Tech Questions (All Candidates)
3. Basic DevOps Questions (30)
4. Medium DevOps Questions (30)
5. Hard DevOps Questions (30)
6. Must-Know Questions for Every DevOps Engineer

Interview Timeline & Structure

Total Duration: 60-90 Minutes

Phase 1: Introduction & Baseline (10-15 minutes)

- Brief introduction and candidate background
- Ask 3-5 baseline tech questions to assess general technical knowledge
- Assess communication skills and attitude

Phase 2: Basic DevOps Concepts (15-20 minutes)

- Ask 8-10 basic questions
- Focus on fundamental understanding
- Assess theoretical knowledge and practical awareness

Phase 3: Medium-Level Technical Assessment (20-25 minutes)

- Ask 8-10 medium questions
- Include scenario-based questions
- Evaluate problem-solving approach
- Assess hands-on experience

Phase 4: Advanced/Hard Questions (15-20 minutes)

- Ask 5-7 hard questions based on candidate's experience level

- Deep dive into specific areas of expertise
- Assess architectural thinking and best practices

Phase 5: Must-Know Questions (5-10 minutes)

- Ask 3-4 critical must-know questions
- These are non-negotiable for any DevOps role

Phase 6: Candidate Questions & Wrap-up (5-10 minutes)

- Allow candidate to ask questions
- Discuss next steps

Baseline Tech Questions

Questions Every Tech Professional Should Know:

1. **What is the OSI model and can you explain at least 4 layers?**
    - Tests fundamental networking knowledge
2. **Explain the difference between TCP and UDP.**
    - Basic protocol understanding
3. **What is DNS and how does it work?**
    - Essential internet infrastructure knowledge
4. **What are the different HTTP status codes? Explain 2xx, 3xx, 4xx, 5xx categories.**
    - Web technology fundamentals
5. **What is the difference between a process and a thread?**
    - Operating system basics
6. **Explain what an API is and give an example of REST API.**
    - Modern application architecture
7. **What is Git and why is it used?**
    - Version control basics
8. **What is the difference between SQL and NoSQL databases? Give examples.**
    - Database fundamentals
9. **What is cloud computing? Name the three main service models.**

- Cloud basics (IaaS, PaaS, SaaS)

10. **Explain what load balancing is and why it's important.**
    - Scalability concepts

Basic DevOps Questions

Q1-10: Core Concepts & Definitions

1. **What is DevOps and why is it important?**
2. **Explain the DevOps lifecycle/pipeline.**
3. **What is CI/CD? Explain both Continuous Integration and Continuous Deployment.**
4. **What is version control and why is it essential in DevOps?**
5. **What is Infrastructure as Code (IaC)?**
6. **Name some popular DevOps tools you're familiar with.**
7. **What is containerization? Why is it useful?**
8. **What is the difference between Agile and DevOps?**
9. **What is configuration management?**
10. **What is monitoring and why is it critical in DevOps?**

Q11-20: Basic Tools & Technologies

11. **What is Docker? Explain its basic architecture.**
12. **What is Kubernetes? Why do we need container orchestration?**
13. **What is Jenkins? What is it used for?**
14. **What is Ansible? How does it differ from other configuration management tools?**
15. **Explain what Git branches are and why they're used.**
16. **What is a Dockerfile?**
17. **What are the basic Linux commands every DevOps engineer should know?**
18. **What is SSH and why is it used?**
19. **What is a firewall?**
20. **What is the purpose of a CI/CD pipeline?**

Q21-30: Basic Scenarios & Concepts

21. **How would you troubleshoot a slow-running application?**

22. **What is the difference between horizontal and vertical scaling?**
23. **What is a microservices architecture?**
24. **What is blue-green deployment?**
25. **What are environment variables and why are they used?**
26. **What is the difference between a container and a virtual machine?**
27. **What is logging and why is it important?**
28. **What is a reverse proxy?**
29. **What does "idempotent" mean in DevOps context?**
30. **What is technical debt and how does DevOps help address it?**

Medium DevOps Questions

Q31-40: Advanced CI/CD & Automation

31. **Explain the difference between declarative and scripted pipelines in Jenkins.**
32. **How would you implement a multi-stage CI/CD pipeline?**
33. **What are Jenkins agents/slaves and when would you use them?**
34. **Explain GitFlow workflow. What are feature branches, release branches, and hotfix branches?**
35. **How do you handle secrets and sensitive data in CI/CD pipelines?**
36. **What is the difference between Continuous Delivery and Continuous Deployment?**
37. **How would you implement automated testing in a CI/CD pipeline?**
38. **What are webhooks and how are they used in CI/CD?**
39. **Explain the concept of pipeline as code.**
40. **How do you implement rollback strategies in your deployment pipeline?**

Q41-50: Container Orchestration & Kubernetes

41. **Explain Kubernetes architecture (control plane and worker nodes).**
42. **What is a Kubernetes Pod? Can a Pod contain multiple containers?**
43. **Explain Kubernetes Services (ClusterIP, NodePort, LoadBalancer).**
44. **What are Kubernetes Deployments and ReplicaSets?**
45. **What is a Kubernetes Namespace and when should you use them?**
46. **Explain ConfigMaps and Secrets in Kubernetes.**
47. **What are Kubernetes Ingress and Ingress Controllers?**

48. **How does Kubernetes handle self-healing and high availability?**
49. **What is the difference between StatefulSet and Deployment in Kubernetes?**
50. **Explain Kubernetes resource requests and limits.**

Q51-60: Monitoring, Logging & Troubleshooting

51. **What are the four golden signals of monitoring?**
52. **Explain the ELK/EFK stack (Elasticsearch, Logstash/Fluentd, Kibana).**
53. **What is Prometheus and how does it work?**
54. **What is Grafana and how is it used with Prometheus?**
55. **How do you implement distributed tracing in microservices?**
56. **What are health checks and readiness probes?**
57. **How would you debug a pod that's in CrashLoopBackOff state?**
58. **Explain different types of logs (application logs, system logs, audit logs).**
59. **What is observability and how is it different from monitoring?**
60. **How do you set up alerts and notification systems?**

Hard DevOps Questions

Q61-70: Architecture & Design Patterns

61. **Design a highly available and scalable architecture for a web application. Explain your choices.**
62. **How would you implement a zero-downtime deployment strategy?**
63. **Explain the concept of immutable infrastructure and its benefits.**
64. **Design a disaster recovery plan. What's your RTO and RPO strategy?**
65. **How do you handle database migrations in a microservices architecture?**
66. **Explain the strangler pattern for migrating from monolith to microservices.**
67. **How would you design a multi-region deployment with failover capabilities?**
68. **Explain the circuit breaker pattern and when you'd use it.**
69. **How do you implement service mesh? What problems does it solve?**
70. **Design a secure CI/CD pipeline that complies with SOC2 or similar standards.**

Q71-80: Security & Best Practices

71. **Explain the principle of least privilege and how you implement it in Kubernetes.**

72. **How do you implement secrets management at scale? Compare different solutions (HashiCorp Vault, AWS Secrets Manager, etc.).**
73. **What are Pod Security Policies/Pod Security Standards in Kubernetes?**
74. **Explain network policies in Kubernetes and provide an example use case.**
75. **How do you implement secure container images? What's container image scanning?**
76. **Explain the concept of "shift-left security." How do you implement it?**
77. **How do you handle vulnerability management in your infrastructure?**
78. **What is mTLS and how would you implement it in a microservices environment?**
79. **Explain the shared responsibility model in cloud computing.**
80. **How do you implement compliance as code?**

Q81-90: Performance & Optimization

81. **How would you optimize Docker image size and build time?**
82. **Explain Kubernetes autoscaling (HPA, VPA, Cluster Autoscaler).**
83. **How do you optimize CI/CD pipeline performance? What techniques have you used?**
84. **Explain the concept of caching in different layers (CDN, application, database).**
85. **How do you handle database connection pooling in a containerized environment?**
86. **What are the trade-offs between using managed services vs. self-hosted solutions?**
87. **How do you implement cost optimization in cloud infrastructure?**
88. **Explain the differences between push-based and pull-based deployment models.**
89. **How would you optimize Kubernetes resource utilization in a multi-tenant cluster?**
90. **Explain the CAP theorem and how it affects your architectural decisions.**

Must-Know Questions

Critical Questions Every DevOps Engineer MUST Answer:

These questions are non-negotiable. Any DevOps engineer should be able to answer these confidently.

1. CI/CD Fundamentals

Question: "Explain a complete CI/CD pipeline from code commit to production deployment. What are the key stages?"

Why it matters: This is the heart of DevOps. If a candidate can't explain this, they don't understand the core workflow.

Expected Answer Should Cover:

- Source control trigger (commit/PR)
- Build stage
- Automated testing (unit, integration)
- Security scanning
- Artifact creation
- Deployment to staging
- Approval gates
- Production deployment
- Monitoring and validation

---

2. Container Technology

Question: "You have an application running in a container that keeps restarting. Walk me through your troubleshooting process."

Why it matters: Containers are fundamental to modern DevOps. Troubleshooting skills are essential.

Expected Answer Should Cover:

- Check container logs: `docker logs <container-id>`
- Check resource constraints (memory/CPU limits)
- Verify health checks
- Check application startup time vs. restart policy
- Examine configuration and environment variables
- Check dependencies and network connectivity
- Review image build process

---

3. Infrastructure as Code

Question: "What is Infrastructure as Code and what are its benefits? Give practical examples."

Why it matters: IaC is a fundamental DevOps principle. Understanding this shows maturity in DevOps practices.

Expected Answer Should Cover:

- Definition: Managing infrastructure through code
- Benefits: Version control, repeatability, consistency, automation
- Tools: Terraform, CloudFormation, Pulumi, Ansible
- Examples: Provisioning VMs, creating networks, configuring security groups
- Idempotency and state management

---

4. Git Workflow

Question: "You accidentally committed sensitive data (like passwords) to a Git repository. What do you do?"

Why it matters: Security awareness and Git proficiency are critical.

Expected Answer Should Cover:

- Immediate actions: Rotate the compromised credentials
- Remove from history: `git filter-branch` or BFG Repo-Cleaner
- Force push to overwrite remote history
- Notify team members to rebase their branches
- Implement preventive measures: pre-commit hooks, secrets scanning
- Consider Git history as permanently compromised

---

5. Monitoring & Incident Response

Question: "Your production application is down. Walk me through your incident response process."

Why it matters: Shows ability to handle pressure and systematic problem-solving.

Expected Answer Should Cover:

- Acknowledge the incident
- Check monitoring dashboards and alerts
- Review recent deployments/changes
- Check infrastructure health (servers, databases, network)
- Review logs and error messages
- Implement immediate fix or rollback
- Communicate with stakeholders
- Post-incident: Root cause analysis and preventive measures

---

6. Linux Systems Administration

Question: "How do you check system performance on a Linux server? What commands would you use?"

Why it matters: Linux is the backbone of most DevOps environments.

Expected Answer Should Cover:

- `top` or `htop` - CPU and memory usage
- `df -h` - Disk space
- `free -h` - Memory usage
- `iostat` - I/O statistics
- `netstat` or `ss` - Network connections
- `uptime` - Load average
- `journalctl` or log files - System logs
- `ps aux` - Process information

---

7. Kubernetes Fundamentals

Question: "Explain the role of each component in Kubernetes architecture."

Why it matters: Kubernetes is the industry standard for container orchestration.

Expected Answer Should Cover:

- **Control Plane:** kube-apiserver, etcd, kube-scheduler, kube-controller-manager
- **Worker Nodes:** kubelet, kube-proxy, container runtime
- How they interact to manage containerized applications
- Basic understanding of desired state vs. actual state

---

8. Security Mindset

Question: "What security considerations do you keep in mind when building a CI/CD pipeline?"

Why it matters: Security should be integrated into DevOps practices.

Expected Answer Should Cover:

- Secrets management (never hardcode credentials)

- Access control and authentication
- Code scanning and vulnerability detection
- Container image scanning
- Least privilege principle
- Audit logging
- Secure artifact storage
- Network security and isolation

Scoring Guide

Junior DevOps Engineer:

- Should answer 80%+ of Basic questions
- Should answer 40-50% of Medium questions
- May struggle with Hard questions (acceptable)
- **MUST answer at least 5-6 of the Must-Know questions**

Mid-Level DevOps Engineer:

- Should answer 90%+ of Basic questions
- Should answer 70%+ of Medium questions
- Should answer 30-40% of Hard questions
- **MUST answer 7-8 of the Must-Know questions confidently**

Senior DevOps Engineer:

- Should answer 95%+ of Basic questions
- Should answer 85%+ of Medium questions
- Should answer 60%+ of Hard questions
- **MUST answer ALL Must-Know questions comprehensively**

Interview Tips

For Interviewers:

1. **Start with easier questions** to make the candidate comfortable

2. **Follow up** on answers to assess depth of knowledge
3. **Allow candidates to think -** don't rush them
4. **Ask about real experiences -** "Tell me about a time when..."
5. **Look for problem-solving approach** rather than just memorized answers
6. **Assess cultural fit** and communication skills
7. **Give practical scenarios** related to your actual tech stack

Red Flags to Watch For:

- Cannot explain basic DevOps concepts
- No hands-on experience with common tools
- Cannot troubleshoot simple problems
- No understanding of security practices
- Poor communication skills
- Cannot explain past project decisions
- Only theoretical knowledge, no practical application

Green Flags to Look For:

- Can explain concepts in simple terms
- Shows systematic problem-solving approach
- Asks clarifying questions
- Admits when they don't know something
- Shows continuous learning attitude
- Can discuss trade-offs in technical decisions
- Has contributed to open source or personal projects
- Understands business impact of technical decisions

Additional Resources for Candidates

Recommended Study Areas:

- Linux fundamentals and shell scripting
- Docker and containerization
- Kubernetes basics and administration

- CI/CD tools (Jenkins, GitLab CI, GitHub Actions)
- Configuration management (Ansible, Terraform)
- Cloud platforms (AWS, Azure, GCP)
- Monitoring and logging (Prometheus, Grafana, ELK)
- Networking basics
- Security best practices
- Scripting languages (Python, Bash, Go)

Hands-on Practice:

- Set up a personal CI/CD pipeline
- Deploy applications on Kubernetes
- Create Infrastructure as Code projects
- Contribute to open source projects
- Build a homelab
- Get cloud certifications

Conclusion

This interview guide provides a comprehensive framework for assessing DevOps candidates at all levels. Remember that the best interviews are conversations, not interrogations. Use these questions as a guide, but be flexible based on the candidate's experience and your organization's specific needs.

Good luck with your interviews!

Document Version: 1.0 Last Updated: February 2026